

STUAART LoadCell + LoadCellController

2.1

Generated by Doxygen 1.17.0

1 LoadCell and LoadCellController libraries for STUAART	1
1.1 Overview	1
1.2 Minimal usage	1
1.3 HX711 corruption handling	2
1.4 References	2
2 STUAART Serial Command Interface	2
2.1 Overview	2
2.2 Destructive command confirmation	3
2.3 Command reference	3
2.4 Typical bench workflow	3
2.5 Implementation notes	4
2.6 Notes on persistence	4
3 System architecture	4
3.1 Overview	4
3.2 Pin mapping (V2 PCB in production)	5
3.3 Boot modes	5
3.4 Main loop	6
3.5 Throughput	6
3.6 CSV data format	7
4 Calibration and the weight equation	7
4.1 The linear model	7
4.2 Why scales can be negative	7
4.3 Typical bench calibration	8
4.4 Persistence and boot behaviour	8
4.5 Manual override	9
4.6 Drift considerations	9
4.7 Mechanical : platform stiffness	10
5 HX711 corruption on STUAART V2 (GPIO 9 / flash bus)	10
5.1 Summary	10
5.2 The shared pad : two drivers on one wire	10
5.3 The three known signatures	11
5.4 safe_read() : retry on corruption	11
5.5 Limits of the software fix	12
5.6 The hardware fix	12
5.7 Diagnosing corruption in CSV files	12
5.8 Live corruption statistics	13
6 Todo List	13
7 Topic Documentation	13

7.1 LoadCell functions for LoadCellController	13
7.1.1 Detailed Description	15
7.1.2 Function Documentation	15
8 Directory Documentation	25
8.1 docs Directory Reference	25
8.2 dox Directory Reference	26
9 Class Documentation	26
9.1 LoadCell Class Reference	26
9.1.1 Detailed Description	28
9.1.2 Constructor & Destructor Documentation	29
9.1.3 Member Function Documentation	29
9.1.4 Member Data Documentation	33
9.2 LoadCellController Class Reference	34
9.2.1 Detailed Description	37
9.2.2 Constructor & Destructor Documentation	38
9.2.3 Member Function Documentation	38
9.2.4 Member Data Documentation	50

1 LoadCell and LoadCellController libraries for STUAART

1.1 Overview

Two custom Arduino classes for the STUAART automated mouse weighing system :

- [LoadCell functions for LoadCellController](#) extends [bogde/HX711](#) with configurable averaging counts for the three usage contexts (weight reading, tare calibration, scale-coefficient calibration) and with a defensive `safe_read()` that filters out three known HX711 corruption signatures (0xFFFFFFFF, 0x800000, 0x7FFFFFFF).
- [LoadCellController](#) manages up to 10 [LoadCell](#) instances per cage, persists calibration to SPIFFS (ESP32) or EEPROM (AVR), and provides ready-to-use start-up routines.

Both classes are compiled directly with the sketch (next to the `.ino`), so cloning the repo and opening `Firmware/STUAART/STUAART.ino` is enough to build the firmware.

1.2 Minimal usage

```
#include "LoadCell.h"
#include "LoadCellController.h"

LoadCell loadcell;
LoadCellController controller;

void setup() {
  Serial.begin(115200);
  controller.add_loadcell(loadcell);
  controller.easy_start_with_params(
    1,      // loadcell number
    27,     // DOUT (avoid GPIO 6-11 on ESP32 : flash bus)
```

```

    17,    // SCK
    true,  // calibrate offset
    true,  // calibrate scale
    false, // read offset from memory
    false, // read scale from memory
    true,  // save offset to memory
    true,  // save scale to memory
    0,     // manual tare offset
    0,     // manual scale coeff
    128    // gain
  );
}

void loop() {
  controller.wait_ready_timeout(1, 1000);
  Serial.println(controller.get_weight(1));
}

```

1.3 HX711 corruption handling

The HX711 24-bit ADC can return three values that are physically impossible during normal operation of a strain-gauge load cell :

- `-1L (0xFFFFF)` : DOUT was held HIGH during the read, typically because an interrupt stretched a SCK pulse beyond 60 us and the chip entered power-down, OR because the DOUT pin shares a pad with the ESP32 internal flash SPI bus (GPIO 6 to 11).
- `-8388608L (0x800000)` : 24-bit negative saturation, observed when the boot tare hits a fully corrupted read on a flash-pin DOUT.
- `+8388607L (0x7FFFFFF)` : 24-bit positive saturation, mirror image.

`LoadCell::safe_read` retries up to three times when any of these appears, and the three averaging methods exclude such samples from the mean rather than averaging garbage in. Callers should treat a returned `-1L` as an invalid reading (returned only if every retry was still corrupt).

For the GPIO 9 case, the software workaround is partial : intermediate bit-patterns from the flash bus can still corrupt occasional reads without matching one of the three signatures. The proper fix is a hardware strap that moves the DOUT off the flash pin.

1.4 References

- bogde/HX711 : <https://github.com/bogde/HX711>
- STUAART repo : <https://github.com/DCC-Lab/STUAART>

Author

Nathan Bérubé, Valérie Pineau Noël, Daniel C. Côté

2 STUAART Serial Command Interface

2.1 Overview

The STUAART firmware exposes an interactive command interface on the USB serial port at 115200 baud. After boot the firmware prints a hint pointing at `help` and waits for input. CSV streaming on Serial is **OFF by default** ; type `stream on` to start it.

The line accumulator is non-blocking : the firmware keeps reading the load cells at 80 Hz while you type. Each command is terminated by `\n` or `\r` (Enter). A typed line is echoed back as `> <cmd>` for confirmation.

2.2 Destructive command confirmation

Commands that modify state (tare, save, reset) ask a `y/n` confirmation on the next line :

```
> tare
confirm 'tare' ? (y/n)
> y
taring all cells
offset cell 1 = -1212208.00
...
```

Any reply other than `y` or `Y` cancels.

2.3 Command reference

Command	Confirm?	Effect
help, ?	no	List all commands
info	no	Show WiFi / SD / RTC / streaming / calibration state
read	no	One immediate read : weights and raws of all 3 cells
raw	no	One immediate read : raw 24-bit signed counts only
stats	no	Per-cell HX711 read counts and corruption rates
reset stats	no	Zero the corruption counters on all 3 cells
stream on	no	Start CSV streaming on Serial (time, w1, w2, w3, raw1, raw2, raw3)
stream off, quiet	no	Stop streaming
tare	yes	Re-tare all 3 cells (reads current raw as new zero)
tare <n>	yes	Re-tare cell n (1, 2 or 3) only
cal <n> <w>	no	Place reference weight w grams on cell n, recompute its scale
set offset <n> <v>	no	Force the offset of cell n to v (RAM only ; use save to persist)
set scale <n> <v>	no	Force the scale of cell n to v (RAM only ; v must be non-zero)
save	yes	Write all offsets and scales to SPIFFS
load	no	Reload all offsets and scales from SPIFFS
wifi	no	Retry WiFi connection (15 s timeout)
time YYYY-MM-DD HH:MM:SS	no	Set the PCF8523 RTC
reset	yes	Soft reboot the firmware

2.4 Typical bench workflow

Calibrate cell 1 with a 100 g reference weight :

```
> tare
confirm 'tare' ? (y/n)
> y
taring all cells
offset cell 1 = -1212208.00

# cells must be empty at this point

# place 100 g on cell 1
> cal 1 100.0
scale cell 1 = -15701.53

> save
```

```
confirm 'save' ? (y/n)
> y
calibration saved to SPIFFS
```

Observe live readings :

```
> stream on
streaming ON
16365,11.22,0.05,-0.02,-2475471,-1212268,-1213545
16372,11.21,0.09, 0.04,-2476234,-1212156,-1213402
...
```

Reset cell 1 to a known offset and scale (e.g. when boot tare was polluted by GPIO 9 corruption) :

```
> set offset 1 -1212208
offset cell 1 = -1212208.00
> set scale 1 -15701.53
scale cell 1 = -15701.53
> save
```

2.5 Implementation notes

- `readSerialLine()` accumulates bytes into a 64-byte buffer `cmdBuf` and returns `true` only when a complete line is received. Called once per `loop()` iteration.
- `executeCommand()` dispatches a finished line. Non-destructive commands run inline ; destructive ones go through `confirmAndRun()` which stores the request in `pendingCmd` and prompts for `y/n`.
- `runImmediate()` executes a (possibly confirmed) command. Used both for direct non-destructive commands and after `y` confirmation.
- `processSerialInput()` is the main entry point called from `loop()`.

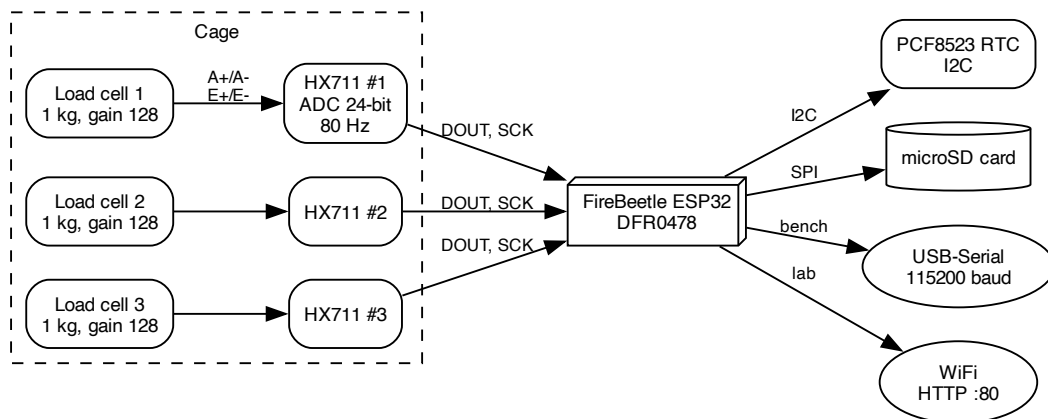
2.6 Notes on persistence

`set offset` and `set scale` modify only the RAM state. Use `save` to persist the values to SPIFFS. Note that the firmware boot path in auto mode currently re-tares (overwriting any saved offset on the next reboot). The scale survives. Future improvement : read offset from SPIFFS at boot if `controller` has a saved value.

3 System architecture

3.1 Overview

STUAAART is built around a single FireBeetle ESP32 V1 (DFRobot DFR0478, based on the Espressif ESP32-D0WD die) that reads three independent load cells, persists each sample to a microSD card, and exposes a minimal HTTP server for remote retrieval. A PCF8523 RTC keeps wall time across power cycles. An interactive Serial command interface (115200 baud) allows on-bench calibration and diagnostics.



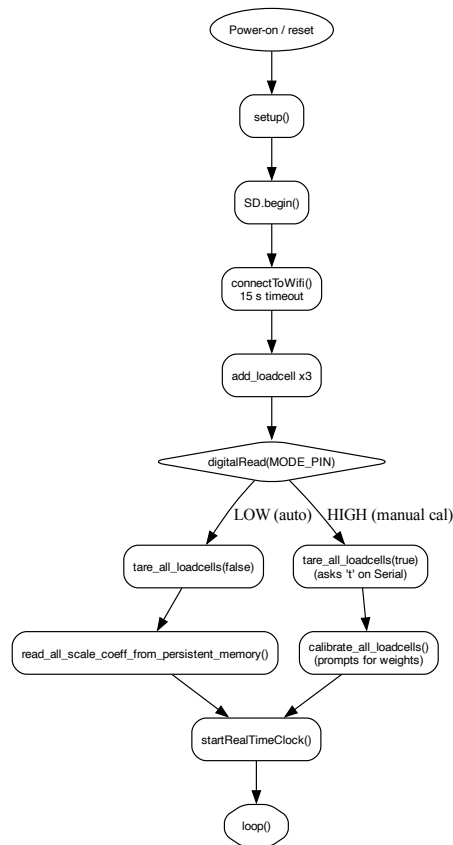
3.2 Pin mapping (V2 PCB in production)

Function	FireBeetle silkscreen	ESP32 GPIO
Cell 1 DOUT	D5	9 (flash bus, see HX711 corruption on STUAART V2 (GPIO 9 / flash bus))
Cell 1 SCK	LRCK	17
Cell 2 DOUT	D4	27
Cell 2 SCK	D1	16
Cell 3 DOUT	D8	5
Cell 3 SCK	D2	25
SD card SS	D8 alias	5
RTC SDA/SCL	SDA / SCL	21 / 22
Mode switch	D7	13
Status LEDs	D6, D9, A4, D8	10, 2, 15, 5

3.3 Boot modes

The mode pin (`MODE_PIN = GPIO 13, INPUT_PULLDOWN`) selects the boot behaviour through the SW1 switch on the PCB :

- **SW1 OFF (LOW = auto)** : the firmware boots quietly, tares all cells live, loads the scale coefficients from SPIFFS, and starts the data acquisition loop. Used in production.
- **SW1 ON (HIGH = manual cal)** : the firmware enters an interactive calibration sequence on Serial, prompting the user to remove the load ('t' to confirm tare), then to place known reference weights for each cell. Persists everything to SPIFFS.



3.4 Main loop

Each iteration of `loop()` runs four steps :

1. `processSerialInput()` reads available bytes from `Serial`. If a full line is ready, it is dispatched to a command handler (see [STUART Serial Command Interface](#)). Non-blocking, so it does not impede the sampling rate.
2. `saveData()` reads each of the 3 cells once via `read_raw_average()`, derives weights via `mass_←from_raw()`, and writes one CSV row to SD (and `Serial`, if `streamOn = true`).
3. Once an hour, `connectToWifi()` is retried so a transient outage does not permanently disable the HTTP server.
4. If a TCP client is connected, the GET line is parsed, the requested CSV is opened from SD (default = today's), and streamed back in 128-byte chunks.

3.5 Throughput

The loop rate is bounded by the HX711 conversion rate, not the CPU. STUART boards have the HX711 RATE pin tied to VCC, configuring the chip for 80 Hz conversions (12.5 ms per sample). With three cells sampled in series and `weight_n_readings = 1`, the loop sustains **~85 Hz** in production. Each row in today's CSV therefore represents ~12 ms of physical time.

3.6 CSV data format

Daily files are named `/YYYY.MM.DD.csv` on the SD card, with a 7-column header :

```
* time (ms), reading 1, reading 2, reading 3, raw 1, raw 2, raw 3
*
```

Each subsequent row is the millisecond timestamp since boot, three weights in grams, and three signed 24-bit raw HX711 counts. Weight and raw on the same row come from the same conversion (see `saveData()`). The raw columns are useful for diagnosing corruption directly without going through the calibration pipeline ; see the `add_binary_columns.py` post-processing script.

4 Calibration and the weight equation

4.1 The linear model

Each load cell + HX711 chain is modeled as a linear function from applied mass to raw 24-bit ADC count :

```
*      raw = scale * mass + offset
*
```

Two parameters per cell, both stored on SPIFFS :

Parameter	Units	What it captures
offset	counts	Raw value at zero load (tare reference)
scale	counts / gram	Sensitivity ; sign depends on wiring polarity

The firmware reverses the equation to derive a weight :

```
*      mass = ( raw - offset ) / scale
*
```

implemented in `LoadCellController::mass_from_raw`. The two values are independent : a wrong `offset` shifts every reading uniformly, a wrong `scale` rotates the response curve.

4.2 Why scales can be negative

`scale` can be either positive or negative depending on the polarity of the load cell wiring. The HX711 measures the differential voltage $V(A+) - V(A-)$. Swapping the A+ and A- leads inverts the sign of every conversion, which translates to a negative `scale` in the linear model.

On STUAART production data we observe :

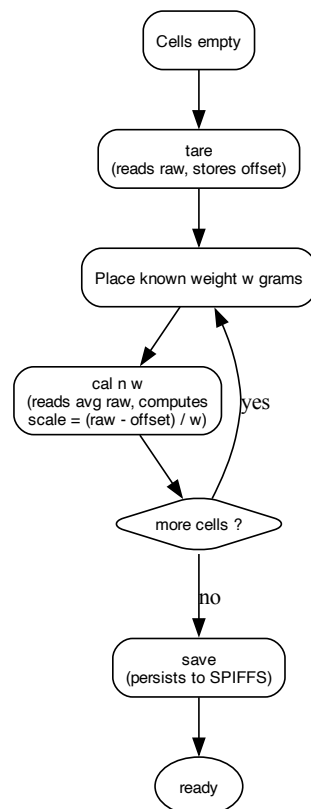
- cell 2 scale 15 700 counts/g
- cell 3 scale 14 700 counts/g

Both negative because the cells were wired with the same convention. Cell 1 in some calibrations shows a positive scale ($\sim +143\,000$) which is anomalous and likely the result of a calibration done on a GPIO-9-corrupted reading.

4.3 Typical bench calibration

The interactive flow uses the [STUART Serial Command Interface](#) command interface :

```
* 1. Empty the cells.
* 2. > tare      (asks confirmation)  -> reads raw, stores as offset
* 3. Place a known reference weight w on cell 1.
* 4. > cal 1 100.0      -> averages 50 reads, computes
*                        scale = (raw - offset) / w
* 5. Repeat steps 3-4 for cells 2 and 3.
* 6. > save      (asks confirmation)  -> writes both to SPIFFS
*
```



4.4 Persistence and boot behaviour

SPIFFS holds two files per cell :

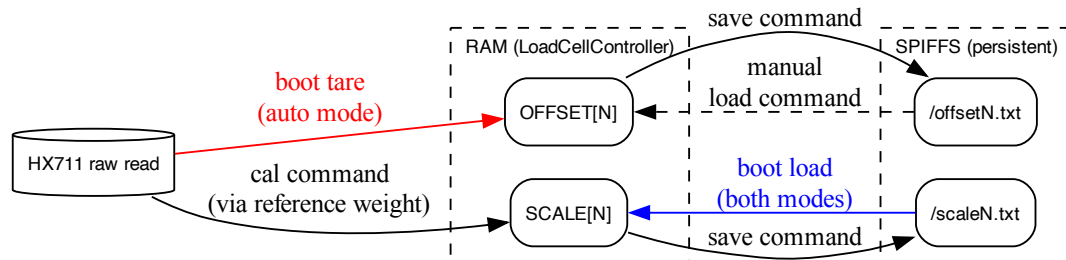
- `/offsetN.txt` : the tare reference for cell N
- `/scaleN.txt` : the scale coefficient for cell N

On a fresh boot in **auto mode** (SW1 OFF), the firmware :

- **re-tares** every cell live by averaging `tare_n_readings = 2` raw reads. This compensates slow drift due to temperature and creep in the printed platforms, but **overwrites** any offset previously persisted to SPIFFS.

- **loads** the scale coefficients from SPIFFS unchanged.

In **manual cal mode** (SW1 ON), the firmware enters the interactive sequence above and persists everything to SPIFFS at the end.



4.5 Manual override

`set offset <n> <v>` and `set scale <n> <v>` modify only the RAM state. Use `save` afterwards to persist. Useful when the boot tare has been polluted by GPIO 9 corruption and you want to force a known reasonable value :

```

> set offset 1 -1212208          # cell 2 was -1212300, similar order
offset cell 1 = -1212208.00
> set scale 1 -15701.53          # match the cell 2 scale
scale cell 1 = -15701.53
> save                          # persist both
  
```

Warning

In auto mode, the saved offset will be overwritten by the next boot tare. The manual override is therefore only a "right now" fix, not a durable one. To make it durable, change the boot path to read the offset from SPIFFS instead of re-taring (one of the improvements catalogued in [Firmware/docs/improvements-to-port.md](#)).

4.6 Drift considerations

The 3D-printed scale platforms creep under sustained load, and the load cells themselves drift with temperature. Production data from STUAART shows ~0.5 g of drift over a 24-hour mouse experiment. Strategies in current use :

- **Re-tare at every boot** (auto mode default) : compensates day-over-day drift but assumes the cage is empty when the firmware boots, which is not always true in practice.
- **Auto-recalibration during the run** : [LoadCellController::get_weight_with_auto_recapibration](#) re-tares any cell that reads below `mouse_weight / 3` for too long. Not currently called from the production sketch but available.
- **Post-processing baseline correction** : subtract a slowly varying baseline from the raw stream offline. See [PostProcessingCode/](#).

4.7 Mechanical : platform stiffness

The CAD note in CAD/ recommends printing the load cell platforms with high infill density to maximize stiffness and minimize creep. A stiffer platform also reduces the effect of mouse position on the platform on the measured weight.

5 HX711 corruption on STUAART V2 (GPIO 9 / flash bus)

5.1 Summary

On the STUAART V2 PCB, cell 1 DOUT is routed to FireBeetle pin **D5**, which maps to **ESP32 GPIO 9**. On the ESP32-D0WD die used by this FireBeetle revision, GPIO 9 is bonded internally to the external SPI flash bus. The flash controller drives that pad on every CPU cache miss, in parallel with the HX711 driving its DOUT to the same node. The two drivers fight, and a fraction of cell 1 reads come out corrupted.

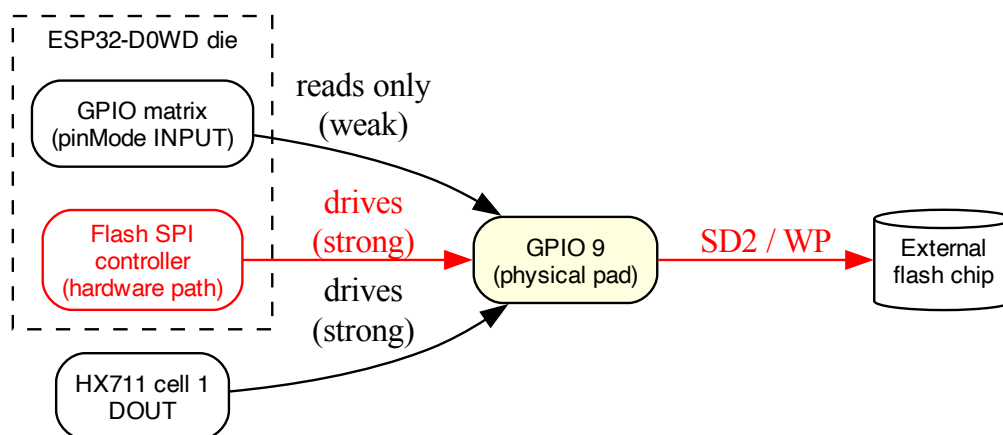
Statistical analysis of the production CSV from 2026-03-10 (4.25 M rows over 13 hours) shows :

Cell	DOUT pin	Outliers	Dominant value	% corruption
1	GPIO 9	2868	-160.68 g	~0.066 %
2	GPIO 27	450	-78.45 g	~0.011 %
3	GPIO 16	< 50	scattered	< 0.001 %

The 160.68 g spike on cell 1 maps exactly to a raw HX711 reading of `0xFFFFFFFF` (= `-1L` after sign extension), the signature of a DOUT line held HIGH throughout the 24-bit shift. ~98 % of cell 1 outliers are this exact value.

5.2 The shared pad : two drivers on one wire

Each pad on the ESP32-D0WD is a multiplexer between several internal functions and the external pin. For GPIO 6 to 11, one of those functions is the dedicated flash SPI controller, hardwired to the pad. `pinMode(9, INPUT)` from the user code only affects the GPIO matrix function ; it does NOT disconnect the flash function, which is implemented in fixed silicon logic.



The flash bus toggles at multi-MHz whenever the CPU executes from flash (which is essentially always). When the user code does `digitalRead(9)` to clock in a HX711 bit, the pad voltage at that instant is whichever driver is winning. If the flash signal is HIGH for the full duration of the shift (which happens during a multi-byte burst), all 24 bits read back as 1 $\rightarrow$ 0xFFFFF.

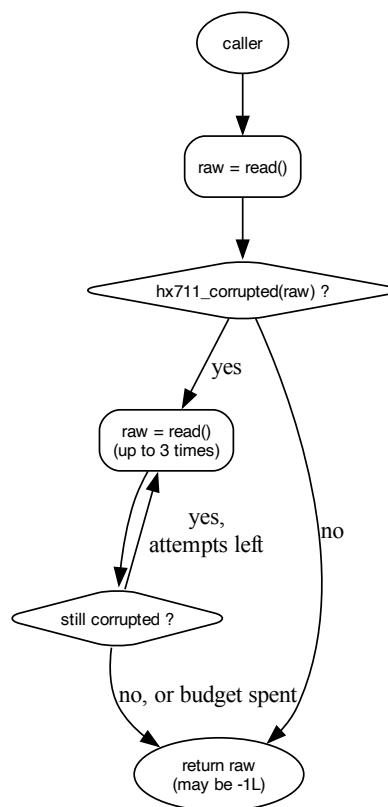
5.3 The three known signatures

The library recognises three values that cannot occur during normal operation of a strain-gauge load cell :

Signature	Meaning	Cause
0xFFFFFFFF (-1L)	DOUT held HIGH throughout the read	Interrupt during SCK pulse > 60 us, OR flash bus contention on GPIO 6-11
0x800000 (-8388608L)	24-bit negative saturation	Boot tare hits a fully-corrupted read
0x7FFFFFFF (+8388607L)	24-bit positive saturation	Mirror image of negative saturation

5.4 `safe_read()` : retry on corruption

`LoadCell::safe_read` calls `read()` and, if the result matches one of the three signatures, retries up to three times. Each retry costs exactly one HX711 conversion (12.5 ms at 80 Hz), so the worst-case overhead per cell is ~ 37 ms. In practice cells 2 and 3 almost never trigger retries, and cell 1 retries succeed on the second attempt about half the time.



The averaging methods (`LoadCell::read_raw_average`, `LoadCell::read_tare_average`, `LoadCell::read_scale_coeff_average`) call `safe_read()` and then exclude any sample that still looks corrupted from the mean. The divisor is the count of GOOD samples, not the requested `n_readings`, so a single bad read does not pull the average toward `0xFFFFFFFF`.

5.5 Limits of the software fix

The three signatures cover ~98 % of cell 1 corruption observed in production. The remaining 2 % are intermediate bit-patterns that happen when the flash bus is partway through a transition during the 24-bit shift : the read returns a plausible-looking value (e.g. `-2 477 564`) that is wrong but indistinguishable from a legitimate reading.

No software primitive can detect these residual cases without external context (e.g. statistical comparison with neighbouring cells, or a median over a sliding window). A median filter over `weight_n_readings = 5` would catch most of them at the cost of dropping the effective sample rate to ~17 Hz — see the discussion in `Firmware/docs/improvements-to-port.md`.

5.6 The hardware fix

The proper resolution is to reroute cell 1 DOUT off GPIO 9. Two pins are unconnected on the V2 PCB and ideal for a strap :

- **D2 = GPIO 25**
- **D3 = GPIO 26**

Recommended procedure :

1. Remove the FireBeetle from its socket on the PCB.
2. Bend the D5 leg outward so it cannot re-enter the socket. This leaves the original PCB trace U1-DAT → D5 dangling but electrically disconnected.
3. Solder a thin wire from the U1-DAT pad (DOUT pin of HX711 #1) to the D2 or D3 socket pin (or directly to the matching FireBeetle pad if more convenient).
4. Re-seat the FireBeetle.
5. In `STUAART.ino`, change `controller.add_loadcell(loadCell1, 9, 17)` to `controller.add_loadcell(loadCell1, 25, 17)` (or `26, 17`).

The strap is reversible (just bend the leg back and remove the wire) and avoids cutting any PCB trace. After rework, cell 1 corruption drops to the same baseline as cells 2 and 3 (~0.01 %, dominated by residual interrupt-induced cases that `safe_read` continues to filter).

5.7 Diagnosing corruption in CSV files

The CSV format includes the raw 24-bit signed counts as the last three columns. To make the corruption signatures jump out, run the post-processing script :

```
python3 PostProcessingCode/add_binary_columns.py today.csv > today_with_bin.csv
```

It appends three columns that show each raw value as its 24-bit two's-complement bit pattern. A row with `111111111111111111111111` in the cell 1 binary column is a `0xFFFFFFFF` corruption signature that escaped `safe_read` (which should not happen post-strap).

5.8 Live corruption statistics

Each `LoadCell` instance keeps four lifetime counters incremented inside `LoadCell::safe_read` :

- `total_reads` : every HX711 conversion consumed
- `corrupted_neg1` : count of `0xFFFFFFFF` (-1L) signatures
- `corrupted_negsat` : count of `0x800000` (-8388608L) signatures
- `corrupted_possat` : count of `0x7FFFFFFF` (+8388607L) signatures

The counters survive across `safe_read` calls but reset on a power-cycle / soft reboot, or whenever the user issues a `reset stats` serial command.

To inspect them at runtime, type `stats` on the serial port :

```
> stats
--- HX711 read statistics ---
cell 1 : reads=85234, 0xFFFFFFFF=56, 0x800000=0, 0x7FFFFFFF=0, total_bad=56 (0.0657 %)
cell 2 : reads=85234, 0xFFFFFFFF=9, 0x800000=0, 0x7FFFFFFF=0, total_bad=9 (0.0106 %)
cell 3 : reads=85234, 0xFFFFFFFF=0, 0x800000=0, 0x7FFFFFFF=0, total_bad=0 (0.0000 %)
```

Use case : compare cells before and after the GPIO 9 strap. Pre-strap cell 1 will show ~0.05-0.07 % corruption (essentially the production baseline) ; post-strap it should drop to the same low level as cells 2 and 3 (< 0.02 %, dominated by residual interrupt-induced cases that `safe_read` continues to filter).

6 Todo List

Member `LoadCellController::easy_start_with_questions` (byte `loadcell_num`, byte `dout`, byte `pd_sck`, byte `gain=128`)

This could be done in the future to simplify the start-up of a `LoadCell` especially for non-programmers.

7 Topic Documentation

7.1 LoadCell functions for LoadCellController

Functions that are the member functions of the `LoadCell` class for the `LoadCellController` class. They are duplicated in the `LoadCellController` class to access the `loadcells` through the `LoadCellController` instance.

Functions

- float `LoadCellController::get_offset` (byte `loadcell_num`)
Get the offset value for a given `LoadCell`.
- void `LoadCellController::set_offset` (byte `loadcell_num`, float `offset`)
Set the offset value for a given `LoadCell`.
- float `LoadCellController::get_scale` (byte `loadcell_num`)
Get the scale coefficient value for a given `LoadCell`.
- void `LoadCellController::set_scale` (byte `loadcell_num`, float `scale`)
Set the scale coefficient value for a given `LoadCell`.
- void `LoadCellController::set_tare_n_readings` (byte `loadcell_num`, int `n_readings`)

- Set the number of readings averaged for calibration of offset for a given [LoadCell](#).

 - byte [LoadCellController::get_tare_n_readings](#) (byte loadcell_num)
- Get the number of readings averaged for calibration of offset for a given [LoadCell](#).

 - void [LoadCellController::set_scale_coeff_n_readings](#) (byte loadcell_num, int n_readings)
- Set the number of readings averaged for calibration of scale coefficient for a given [LoadCell](#).

 - byte [LoadCellController::get_scale_coeff_n_readings](#) (byte loadcell_num)
- Get the number of readings averaged for calibration of scale calibration for a given [LoadCell](#).

 - void [LoadCellController::set_weight_n_readings](#) (byte loadcell_num, int n_readings)
- Set the number of readings averaged for weight measurements for a given [LoadCell](#).

 - byte [LoadCellController::get_weight_n_readings](#) (byte loadcell_num)
- Get the number of readings averaged for measurements of a given [LoadCell](#)'s weight.

 - void [LoadCellController::set_all_loadcells_tare_n_readings](#) (int n_readings)
- Set the number of readings averaged for calibration of offset for all [LoadCells](#).

 - void [LoadCellController::set_all_loadcells_scale_coeff_n_readings](#) (int n_readings)
- Set the number of readings averaged for calibration of scale coefficient for all [LoadCells](#).

 - void [LoadCellController::set_all_loadcells_weight_n_readings](#) (int n_readings)
- Set the number of readings averaged for weight measurements for all [LoadCells](#).

 - void [LoadCellController::set_mouse_weight](#) (float weight)
- Set the value of member variable [mouse_weight](#).

 - float [LoadCellController::get_mouse_weight](#) ()
- Get the value of member variable [mouse_weight](#).

 - long [LoadCellController::read_raw_average](#) (byte loadcell_num)
- Read the output of the [LoadCell](#) and average [weight_n_readings](#) readings.

 - long [LoadCellController::read_tare_average](#) (byte loadcell_num)
- Read the output of the [LoadCell](#) and average [tare_n_readings](#) readings.

 - long [LoadCellController::read_scale_coeff_average](#) (byte loadcell_num)
- Read the output of the [LoadCell](#) and average [scale_coeff_n_readings](#) readings.

 - double [LoadCellController::get_raw_value](#) (byte loadcell_num)
- Read the raw output of the [LoadCell](#) and subtracts the offset.

 - float [LoadCellController::get_weight](#) (byte loadcell_num)
- Read the weight of the object currently on the [LoadCell](#).

 - float [LoadCellController::get_weight_with_auto_recabibration](#) (byte loadcell_num, float threshold=0.33)
- Read the weight of the object currently on the [LoadCell](#) and recalibrate the offset.

 - float [LoadCellController::mass_from_raw](#) (byte loadcell_num, long raw)
- Calculate mass from a raw reading.

 - void [LoadCellController::tare](#) (byte loadcell_num)
- Read the raw output of the [LoadCell](#) and set this value to the offset.

 - void [LoadCellController::begin](#) (byte loadcell_num, byte dout, byte pd_sck, byte gain=128)
- Begin the communication with the HX711 for a given [LoadCell](#).

 - bool [LoadCellController::is_ready](#) (byte loadcell_num)
- Verify if the HX711 is ready for a reading.

 - void [LoadCellController::wait_ready](#) (byte loadcell_num, unsigned long delay_ms=0)
- Suspend the execution of the code until the HX711 becomes ready.

 - bool [LoadCellController::wait_ready_retry](#) (byte loadcell_num, int retries=3, unsigned long delay_ms=0)
- Suspend the execution of the code until the HX711 becomes ready or for a given number of retries.

 - bool [LoadCellController::wait_ready_timeout](#) (byte loadcell_num, unsigned long timeout=1000, unsigned long delay_ms=0)
- Suspend the execution of the code until the HX711 becomes ready or for a given timeout.

 - void [LoadCellController::power_up](#) (byte loadcell_num)
- Wake up the chip after power down mode.

 - void [LoadCellController::power_down](#) (byte loadcell_num)
- Wake up the chip after power down mode.

7.1.1 Detailed Description

Functions that are the member functions of the [LoadCell](#) class for the [LoadCellController](#) class. They are duplicated in the [LoadCellController](#) class to access the [loadcells](#) through the [LoadCellController](#) instance.

Note

[LoadCell](#) class member functions can also be accessed through the [LoadCell](#) instance. The two possibilities are available.

7.1.2 Function Documentation

begin()

```
void LoadCellController::begin (
    byte loadcell_num,
    byte dout,
    byte pd_sck,
    byte gain = 128)
```

Begin the communication with the HX711 for a given [LoadCell](#).

This function begins the communication with the HX711 that is performing a voltage reading of the Wheatstone bridge on the load cell to convert it in a 24-bits digital raw output. This function needs to be called before performing measurements of weight.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>dout</i>	Digital pin (or analog) connected to the DOUT output pin of the HX711.
<i>pd_sck</i>	Digital pin (or analog) connected to the SCK output of the HX711.
<i>gain</i>	Gain of the HX711. Default is 128.

The *gain* argument can take the following values: 128, 64 and 32. For a gain of 128 and 64, channel A of the HX711 should be used. For a gain of 32, channel B should be used.

get_mouse_weight()

```
float LoadCellController::get_mouse_weight ()
```

Get the value of member variable [mouse_weight](#).

This function gets the weight of the mouse in the cage controlled by the [LoadCellController](#).

Returns

Weight of the mouse.

get_offset()

```
float LoadCellController::get_offset (
    byte loadcell_num)
```

Get the offset value for a given [LoadCell](#).

This function gets the offset member variable for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

Offset value.

get_raw_value()

```
double LoadCellController::get_raw_value (
    byte loadcell_num)
```

Read the raw output of the [LoadCell](#) and subtracts the offset.

This function reads the average of readings of the [LoadCell](#) raw output with [read_raw_average\(\)](#) then subtracts the offset accessed with `HX711::get_offset()`.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

The raw averaged reading of the [LoadCell](#) difference with the offset.

get_scale()

```
float LoadCellController::get_scale (
    byte loadcell_num)
```

Get the scale coefficient value for a given [LoadCell](#).

This function gets the scale coefficient member variable for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

Scale coefficient value.

get_scale_coeff_n_readings()

```
byte LoadCellController::get_scale_coeff_n_readings (
    byte loadcell_num)
```

Get the number of readings averaged for calibration of scale calibration for a given [LoadCell](#).

This function sets the number of readings averaged for calibration of offset for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

Number of readings averaged for scale coefficient calibration.

get_tare_n_readings()

```
byte LoadCellController::get_tare_n_readings (
    byte loadcell_num)
```

Get the number of readings averaged for calibration of offset for a given [LoadCell](#).

This function sets the number of readings averaged for calibration of offset for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

Number of readings averaged for offset calibration.

get_weight()

```
float LoadCellController::get_weight (
    byte loadcell_num)
```

Read the weight of the object currently on the [LoadCell](#).

This function reads the average of readings of the [LoadCell](#) raw output with [read_raw_average\(\)](#) before converting this average value to a mass value using the calibration function -> $\text{output} = \text{scale_coeff} * \text{mass} + \text{offset}$.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

The averaged mass of the object on the [LoadCell](#).

get_weight_n_readings()

```
byte LoadCellController::get_weight_n_readings (
    byte loadcell_num)
```

Get the number of readings averaged for measurements of a given [LoadCell](#)'s weight.

This function retrieves the number of readings that are averaged for measurements of a given [LoadCell](#)'s weight.

Parameters

<i>loadcell_num</i>	loadcell_num Number of the LoadCell .
---------------------	---

Returns

Number of reading averaged for a weight measurements.

get_weight_with_auto_recalibration()

```
float LoadCellController::get_weight_with_auto_recalibration (
    byte loadcell_num,
    float threshold = 0.33)
```

Read the weight of the object currently on the [LoadCell](#) and recalibrate the offset.

This member function can be used to perform a weight reading of an object on the [LoadCell](#) will verifying if no object is on the [LoadCell](#). If it is the case, a recalibration of the [LoadCell](#) offset will be done. The threshold to termine if the [LoadCell](#) is empty is calculated from the [mouse_weight](#) and the `threshold`. If their is something on the [LoadCell](#), a weight reading is normally performed. The function returns the reading value in both case, but if a recalibration is done, the reading will most likely be 0g.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>threshold</i>	Fraction of mouse_weight used as the threshold to determine if the LoadCell is empty.

Returns

The averaged mass of the object on the [LoadCell](#).

is_ready()

```
bool LoadCellController::is_ready (
    byte loadcell_num)
```

Verify if the HX711 is ready for a reading.

This function verifies if the HX711 is ready for retrieval the next reading. According to the datasheet of the HX711, when output data is not ready for retrieval, digital output pin DOUT is high. Serial clock input PD_SCK should be low. When DOUT goes to low, it indicates data is ready for retrieval. The latter condition is verified in this member function.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

Boolean showing if the HX711 is ready for retrieval.

mass_from_raw()

```
float LoadCellController::mass_from_raw (
    byte loadcell_num,
    long raw)
```

Calculate mass from a raw reading.

This member function takes as an input a raw reading and calculates the mass associated to it. The calculation requires the scale coefficient and the offset of the [LoadCell](#).

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>raw</i>	Raw reading.

Returns

Mass associated to raw measurement

power_down()

```
void LoadCellController::power_down (
    byte loadcell_num)
```

Wake up the chip after power down mode.

This function powers down the HX711 after calling [power_up\(\)](#) member function.

Note

This function doesn't stop the alimentation of the Wheatstone on the load cell, only suspend the HX711.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

power_up()

```
void LoadCellController::power_up (
    byte loadcell_num)
```

Wake up the chip after power down mode.

This function powers up the HX711 after calling [power_down\(\)](#) member function.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

read_raw_average()

```
long LoadCellController::read_raw_average (
    byte loadcell_num)
```

Read the output of the [LoadCell](#) and average `weight_n_readings` readings.

This function does `weight_n_readings` readings of the raw output of the [LoadCell](#) and calculates their average before returning the average raw output.

Parameters

<code>loadcell_num</code>	Number of the LoadCell .
---------------------------	--

Returns

The raw averaged reading of the [LoadCell](#).

read_scale_coeff_average()

```
long LoadCellController::read_scale_coeff_average (
    byte loadcell_num)
```

Read the output of the [LoadCell](#) and average `scale_coeff_n_readings` readings.

This function does `scale_coeff_n_readings` readings of the raw output of the [LoadCell](#) and calculates their average before returning the average raw output. It is useful inside calibration scripts.

Parameters

<code>loadcell_num</code>	Number of the LoadCell .
---------------------------	--

Returns

The raw averaged reading of the [LoadCell](#) for the scale coefficient calibration.

read_tare_average()

```
long LoadCellController::read_tare_average (
    byte loadcell_num)
```

Read the output of the [LoadCell](#) and average `tare_n_readings` readings.

This function does `tare_n_readings` readings of the raw output of the [LoadCell](#) and calculates their average before returning the average raw output. It is useful inside calibration scripts.

Parameters

<code>loadcell_num</code>	Number of the LoadCell .
---------------------------	--

Returns

The raw averaged reading of the [LoadCell](#) for the tare offset calibration.

set_all_loadcells_scale_coeff_n_readings()

```
void LoadCellController::set_all_loadcells_scale_coeff_n_readings (
    int n_readings)
```

Set the number of readings averaged for calibration of scale coefficient for all LoadCells.

This function sets the number of readings to be averaged for calibration of scale coefficient for all LoadCells.

Parameters

<i>n_readings</i>	Number of readings averaged for calibration of scale coefficient.
-------------------	---

set_all_loadcells_tare_n_readings()

```
void LoadCellController::set_all_loadcells_tare_n_readings (
    int n_readings)
```

Set the number of readings averaged for calibration of offset for all LoadCells.

This function sets the number of readings to be averaged for calibration of offset for all LoadCells.

Parameters

<i>n_readings</i>	Number of readings averaged for calibration of offset.
-------------------	--

set_all_loadcells_weight_n_readings()

```
void LoadCellController::set_all_loadcells_weight_n_readings (
    int n_readings)
```

Set the number of readings averaged for weight measurements for all LoadCells.

This function sets the number of readings to be averaged for weight measurements for all LoadCells.

Parameters

<i>n_readings</i>	Number of readings averaged for weight measurements.
-------------------	--

set_mouse_weight()

```
void LoadCellController::set_mouse_weight (
    float weight)
```

Set the value of member variable [mouse_weight](#).

This function sets the weight of the mouse in the cage controlled by the [LoadCellController](#). This function will only set the input to the member variable [mouse_weight](#) if it is positive. If the input is negative, the default value is kept.

Parameters

<i>weight</i>	Weight of the mouse.
---------------	----------------------

set_offset()

```
void LoadCellController::set_offset (
    byte loadcell_num,
    float offset)
```

Set the offset value for a given [LoadCell](#).

This function sets the offset member variable for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>offset</i>	Offset value.

set_scale()

```
void LoadCellController::set_scale (
    byte loadcell_num,
    float scale)
```

Set the scale coefficient value for a given [LoadCell](#).

This function sets the scale coefficient member variable for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>scale</i>	Scale coefficient value.

set_scale_coeff_n_readings()

```
void LoadCellController::set_scale_coeff_n_readings (
    byte loadcell_num,
    int n_readings)
```

Set the number of readings averaged for calibration of scale coefficient for a given [LoadCell](#).

This function sets the number of readings averaged for calibration of scale coefficient for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>n_readings</i>	Number of readings.

set_tare_n_readings()

```
void LoadCellController::set_tare_n_readings (
    byte loadcell_num,
    int n_readings)
```

Set the number of readings averaged for calibration of offset for a given [LoadCell](#).

This function sets the number of readings averaged for calibration of offset for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

<i>n_readings</i>	Number of readings.
-------------------	---------------------

set_weight_n_readings()

```
void LoadCellController::set_weight_n_readings (
    byte loadcell_num,
    int n_readings)
```

Set the number of readings averaged for weight measurements for a given [LoadCell](#).

This function sets the number of readings to be averaged for weight measurements for a given [LoadCell](#) through the controller.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>n_readings</i>	Number of reading averaged for a weight measurements.

tare()

```
void LoadCellController::tare (
    byte loadcell_num)
```

Read the raw output of the [LoadCell](#) and set this value to the offset.

This function does tare_n_reading readings to get the average for the offset. It then sets this value to the member variable OFFSET.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

wait_ready()

```
void LoadCellController::wait_ready (
    byte loadcell_num,
    unsigned long delay_ms = 0)
```

Suspend the execution of the code until the HX711 becomes ready.

This function suspends the execution of the code with a while loop relying on the [is_ready\(\)](#) condition. When called, this function will stop temporarily the code execution until the HX711 becomes ready.

Warning

Using this function can completely stop the code execution if a technical problem happens with a HX711. It is safer to handle the retrieval of a reading with [wait_ready_retry\(\)](#) or [wait_ready_timeout\(\)](#)

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

<i>delay_ms</i>	Delay between two readiness checks of the HX711, in milliseconds. Default is 0.
-----------------	---

wait_ready_retry()

```
bool LoadCellController::wait_ready_retry (
    byte loadcell_num,
    int retries = 3,
    unsigned long delay_ms = 0)
```

Suspend the execution of the code until the HX711 becomes ready or for a given number of retries.

This function suspends the execution of the code with a while loop relying on the [is_ready\(\)](#) condition. When called, this function will stop temporarily the code execution until the HX711 becomes ready or for a given number of retries. This prevents from a endless suspension of the code.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>retries</i>	Number of retries to verify the "readiness state" of the HX711. Default is 3.
<i>delay_ms</i>	Delay between two verification of the "readiness state" of the HX711. Default is 0.

wait_ready_timeout()

```
bool LoadCellController::wait_ready_timeout (
    byte loadcell_num,
    unsigned long timeout = 1000,
    unsigned long delay_ms = 0)
```

Suspend the execution of the code until the HX711 becomes ready or for a given timeout.

This function suspends the execution of the code with a while loop relying on the [is_ready\(\)](#) condition. When called, this function will stop temporarily the code execution until the HX711 becomes ready or for a given timeout. If after the timeout the HX711 is not ready for retrieval, the code execution will continue. This prevents from a endless suspension of the code.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>timeout</i>	Timeout of the suspension of execution of the code in milliseconds. Default is 1000 ms.
<i>delay_ms</i>	Delay between two verification of the "readiness state" of the HX711 in milliseconds. Default is 0.

8 Directory Documentation

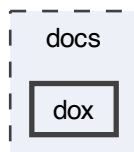
8.1 docs Directory Reference

Directories

- directory [dox](#)

8.2 dox Directory Reference

Directory dependency graph for dox:



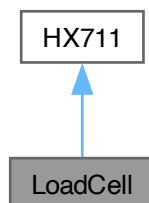
9 Class Documentation

9.1 LoadCell Class Reference

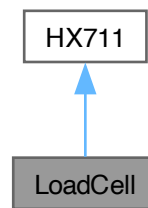
This library extends the functionality of the HX711 to make it compatible with the [LoadCellController](#) library.

```
#include <LoadCell.h>
```

Inheritance diagram for LoadCell:



Collaboration diagram for LoadCell:



Public Member Functions

- `LoadCell ()`
Construct a `LoadCell` object.
- `long safe_read (byte max_retries=3)`
Read the HX711 with corruption detection and retry.
- `unsigned long total_corrupted () const`
Sum of the three per-signature corruption counters.
- `void reset_stats ()`
Zero all four read-statistics counters.
- `long read_raw_average ()`
Read the output of the `LoadCell` and average `weight_n_readings` readings.
- `long read_tare_average ()`
Read the output of the `LoadCell` and average `tare_n_readings` readings.
- `long read_scale_coeff_average ()`
Read the output of the `LoadCell` and average `scale_coeff_n_readings` readings.
- `double get_raw_value ()`
Read the raw output of the `LoadCell` and subtracts the offset.
- `float get_weight ()`
Read the weight of the object currently on the `LoadCell`.
- `void tare ()`
Read the raw output of the `LoadCell` and set this value to the offset.
- `void set_tare_n_readings (int tare_n_readings)`
Set the number of readings averaged for the determination of the tare offset.
- `int get_tare_n_readings ()`
Get the number of readings averaged for the determination of the tare offset.
- `void set_scale_coeff_n_readings (int scale_coeff_n_readings)`
Set the number of readings averaged for the determination of the scale coefficient.
- `int get_scale_coeff_n_readings ()`
Get the number of readings averaged for the determination of the scale coefficient.
- `void set_weight_n_readings (int weight_n_readings)`
Set the number of readings averaged for a weight reading.
- `int get_weight_n_readings ()`
Get the number of readings averaged for a weight reading.

Public Attributes

- unsigned long `total_reads` = 0
Lifetime counter of HX711 conversions consumed by `safe_read` (initial reads + retries).
- unsigned long `corrupted_neg1` = 0
Lifetime count of reads that returned `0xFFFFFFFF` (-1L). Dominant signature on STUAART V2 cell 1 due to GPIO 9 / flash bus contention.
- unsigned long `corrupted_negsat` = 0
Lifetime count of reads that returned `0x800000` (-8388608L, 24-bit negative saturation).
- unsigned long `corrupted_possat` = 0
Lifetime count of reads that returned `0x7FFFFFFF` (+8388607L, 24-bit positive saturation).

Protected Attributes

- byte `tare_n_readings` = 50
Number of readings averaged to determine the tare offset.
- byte `scale_coeff_n_readings` = 50
Number of readings averaged to determine the scale coefficient.
- byte `weight_n_readings` = 5
Number of readings averaged for a weight reading.

9.1.1 Detailed Description

This library extends the functionality of the HX711 to make it compatible with the [LoadCellController](#) library.

This Arduino library allows to specify the number of readings averaged for the three possible situations:

- Weight reading
- Output reading for offset calibration
- Output reading for scale coefficient calibration

The encapsulation of these parameters requires the rewriting of a few member functions of the HX711 library. They do the same thing but, with the encapsulation of the three parameters listed above, they don't require any arguments.

Most importantly, the encapsulation of these parameters is useful for the [LoadCellController](#) library, which will control many LoadCells.

HX711 library: <https://github.com/bogde/HX711>

[LoadCellController](#) library: <https://github.com/DCC-Lab/IntelligentCage/tree/master/Loadcell/Arduino%20libraries/LoadCellControllerlibrary>

Author

Nathan Bérubé

Date

August 14, 2023

Version

1.0.0

9.1.2 Constructor & Destructor Documentation

LoadCell()

```
LoadCell::LoadCell ()
```

Construct a [LoadCell](#) object.

9.1.3 Member Function Documentation

get_raw_value()

```
double LoadCell::get_raw_value ()
```

Read the raw output of the [LoadCell](#) and subtracts the offset.

This function reads the average of readings of the [LoadCell](#) raw output with [read_raw_average\(\)](#) then subtracts the offset accessed with `HX711::get_offset()`.

Returns

The raw averaged reading of the [LoadCell](#) difference with the offset.

get_scale_coeff_n_readings()

```
int LoadCell::get_scale_coeff_n_readings ()
```

Get the number of readings averaged for the determination of the scale coefficient.

This function returns the member variable [scale_coeff_n_readings](#).

Returns

The number of readings averaged for the determination of the tare offset.

get_tare_n_readings()

```
int LoadCell::get_tare_n_readings ()
```

Get the number of readings averaged for the determination of the tare offset.

This function returns the member variable [tare_n_readings](#).

Returns

The number of readings averaged for the determination of the tare offset.

get_weight()

```
float LoadCell::get_weight ()
```

Read the weight of the object currently on the [LoadCell](#).

This function reads the average of readings of the [LoadCell](#) raw output with [read_raw_average\(\)](#) before converting this average of raw output to a mass value using the calibration function -> $\text{output} = \text{scale_coeff} * \text{mass} + \text{offset}$.

Returns

The averaged mass of the object on the [LoadCell](#).

get_weight_n_readings()

```
int LoadCell::get_weight_n_readings ()
```

Get the number of readings averaged for a weight reading.

This function returns the member variable [scale_coeff_n_readings](#).

Returns

The number of readings averaged for the determination of the tare offset.

read_raw_average()

```
long LoadCell::read_raw_average ()
```

Read the output of the [LoadCell](#) and average [weight_n_readings](#) readings.

This function does [weight_n_readings](#) readings of the raw output of the [LoadCell](#) and calculates their average before returning the average raw output.

Returns

The raw averaged reading of the [LoadCell](#).

read_scale_coeff_average()

```
long LoadCell::read_scale_coeff_average ()
```

Read the output of the [LoadCell](#) and average [scale_coeff_n_readings](#) readings.

This function does [scale_coeff_n_readings](#) readings of the raw output of the [LoadCell](#) and calculates their average before returning the average raw output. It is useful inside calibration scripts.

Returns

The raw averaged reading of the [LoadCell](#) for the scale coefficient calibration.

read_tare_average()

```
long LoadCell::read_tare_average ()
```

Read the output of the [LoadCell](#) and average [tare_n_readings](#) readings.

This function does [tare_n_readings](#) readings of the raw output of the [LoadCell](#) and calculates their average before returning the average raw output. It is useful inside calibration scripts.

Returns

The raw averaged reading of the [LoadCell](#) for the tare offset calibration.

reset_stats()

```
void LoadCell::reset_stats () [inline]
```

Zero all four read-statistics counters.

safe_read()

```
long LoadCell::safe_read (
    byte max_retries = 3)
```

Read the HX711 with corruption detection and retry.

Rejects three known corruption signatures and retries up to `max_retries` times: 0xFFFFFFF (-1, DOUT held HIGH by interrupt or flash bus contention on GPIO 6-11), 0x800000 (-8388608, negative saturation), and 0x7FFFFFFF (+8388607, positive saturation). None of these can occur during normal operation of a mouse-scale load cell. Returns the last raw value; callers should treat -1L as invalid (returned only if every retry was still corrupt).

Increments [total_reads](#) on every internal `read()` call, and the appropriate `corrupted_*` counter every time a corruption signature is observed (initial read or any retry).

set_scale_coeff_n_readings()

```
void LoadCell::set_scale_coeff_n_readings (
    int scale_coeff_n_readings)
```

Set the number of readings averaged for the determination of the scale coefficient.

This function sets the member variable `scale_coeff_n_readings`. It also limits the range of value that is possible for this member variable -> [scale_coeff_n_readings](#) in [1, 255].

Anything below 1 will be set to 1 and anything above 255 will be set to 255 to be able to store in a byte.

Parameters

scale_coeff_n_readings	The number of readings averaged for the determination of the tare offset.
--	---

set_tare_n_readings()

```
void LoadCell::set_tare_n_readings (
    int tare_n_readings)
```

Set the number of readings averaged for the determination of the tare offset.

This function sets the member variable `tare_n_readings`. It also limits the range of value that is possible for this member variable -> `tare_n_readings` in [1, 255].

Anything below 1 will be set to 1 and anything above 255 will be set to 255 to be able to store in a byte.

Parameters

<code>tare_n_readings</code>	The number of readings averaged for the determination of the tare offset.
------------------------------	---

set_weight_n_readings()

```
void LoadCell::set_weight_n_readings (
    int weight_n_readings)
```

Set the number of readings averaged for a weight reading.

This function sets the member variable `weight_n_readings`. It also limits the range of value that is possible for this member variable -> `weight_n_readings` in [1, 255].

Anything below 1 will be set to 1 and anything above 255 will be set to 255 to be able to store in a byte.

Parameters

<code>weight_n_readings</code>	The number of readings averaged for the determination of the tare offset.
--------------------------------	---

tare()

```
void LoadCell::tare ()
```

Read the raw output of the [LoadCell](#) and set this value to the offset.

This function does `tare_n_reading` readings to get the average for the offset. It then sets this value to the member variable `OFFSET` that is private.

total_corrupted()

```
unsigned long LoadCell::total_corrupted () const [inline]
```

Sum of the three per-signature corruption counters.

Returns

`corrupted_neg1` + `corrupted_negsat` + `corrupted_possat`.

9.1.4 Member Data Documentation

corrupted_neg1

```
unsigned long LoadCell::corrupted_neg1 = 0
```

Lifetime count of reads that returned 0xFFFFF (-1L). Dominant signature on STUAART V2 cell 1 due to GPIO 9 / flash bus contention.

corrupted_negsat

```
unsigned long LoadCell::corrupted_negsat = 0
```

Lifetime count of reads that returned 0x800000 (-8388608L, 24-bit negative saturation).

corrupted_possat

```
unsigned long LoadCell::corrupted_possat = 0
```

Lifetime count of reads that returned 0x7FFFFFFF (+8388607L, 24-bit positive saturation).

scale_coeff_n_readings

```
byte LoadCell::scale_coeff_n_readings = 50 [protected]
```

Number of readings averaged to determine the scale coefficient.

tare_n_readings

```
byte LoadCell::tare_n_readings = 50 [protected]
```

Number of readings averaged to determine the tare offset.

total_reads

```
unsigned long LoadCell::total_reads = 0
```

Lifetime counter of HX711 conversions consumed by [safe_read](#) (initial reads + retries).

Use as denominator when computing the corruption rate : $\text{rate} = \frac{\text{total_corrupted}()}{\text{total_reads}}$

weight_n_readings

```
byte LoadCell::weight_n_readings = 5 [protected]
```

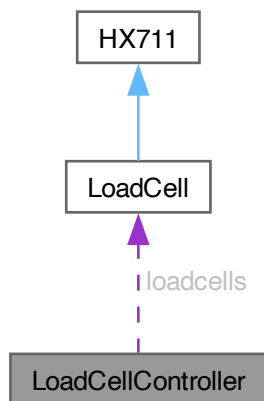
Number of readings averaged for a weight reading.

9.2 LoadCellController Class Reference

This library allows to control and manage many load cells with a [LoadCellController](#).

```
#include <LoadCellController.h>
```

Collaboration diagram for LoadCellController:



Public Member Functions

- [LoadCellController](#) ()
Construct a [LoadCellController](#) object.
- void [add_loadcell](#) ([LoadCell](#) &loadcell)
Add a new [LoadCell](#) to the [LoadCellController](#) without pin specification.
- void [add_loadcell](#) ([LoadCell](#) &loadcell, byte dout, byte sck, byte gain=128)
Add a new [LoadCell](#) to the [LoadCellController](#) with pin specification.
- void [easy_start_with_params](#) (byte loadcell_num, byte dout, byte pd_sck, bool calibrate_offset=false, bool calibrate_scale=false, bool read_offset_persistent_memory=false, bool read_scale_persistent_memory=false, bool save_offset_persistent_memory=false, bool save_scale_persistent_memory=false, float tare_offset=0, float scale_coeff=0, byte gain=128)
Easy function managing the start-up of a [LoadCell](#) with parameters as input.
- void [tare_all_loadcells](#) (bool wait_for_user=true)
Tare all the [LoadCells](#) added to the [LoadCellController](#).
- void [calibrate_all_loadcells](#) ()
Calibrate the scale coefficient of every [LoadCells](#) added to the [LoadCellController](#).
- void [read_all_scale_coeff_from_persistent_memory](#) ()
Read the scale coefficient of every [LoadCells](#) added to the [LoadCellController](#).
- void [save_offset_to_persistent_memory](#) (byte loadcell_num)
Save the offset of a given [LoadCell](#) to persistent memory.
- void [save_scale_coeff_to_persistent_memory](#) (byte loadcell_num)
Save the scale coefficient of a given [LoadCell](#) to persistent memory.
- long [read_offset_from_persistent_memory](#) (byte loadcell_num)
Read the offset of a given [LoadCell](#) from persistent memory.

- float [read_scale_coeff_from_persistent_memory](#) (byte loadcell_num)
Read the scale coefficient of a given [LoadCell](#) to persistent memory.
- void [calibrate_both_params](#) (byte loadcell_num)
Proceed to the calibration of both parameters and set them for a [LoadCell](#).
- void [calibrate_tare_offset](#) (byte loadcell_num)
Proceed to the calibration of the offset and set it for a [LoadCell](#).
- void [calibrate_scale_coeff](#) (byte loadcell_num)
Proceed to the calibration of the scale coefficient and set it for a [LoadCell](#).
- float [determine_offset](#) (byte loadcell_num)
Proceed to the calibration of the offset and return the offset value.
- float [determine_scale_coeff](#) (byte loadcell_num)
Proceed to the calibration of the scale coefficient and return the scale coefficient value.
- float [calculate_scale_coeff](#) (byte loadcell_num, float output, float mass)
- int [get_offset_eeprom_address](#) (byte loadcell_num)
Get the EEPROM address for the offset of a [LoadCell](#).
- int [get_scale_coeff_eeprom_address](#) (byte loadcell_num)
Get the EEPROM address for the scale coefficient of a [LoadCell](#).
- String [get_offset_file_name](#) (byte loadcell_num)
Get the SPIFFS file name for the offset of a [LoadCell](#).
- String [get_scale_coeff_file_name](#) (byte loadcell_num)
Get the SPIFFS file name for the scale coefficient of a [LoadCell](#).
- float [get_offset](#) (byte loadcell_num)
Get the offset value for a given [LoadCell](#).
- void [set_offset](#) (byte loadcell_num, float offset)
Set the offset value for a given [LoadCell](#).
- float [get_scale](#) (byte loadcell_num)
Get the scale coefficient value for a given [LoadCell](#).
- void [set_scale](#) (byte loadcell_num, float scale)
Set the scale coefficient value for a given [LoadCell](#).
- void [set_tare_n_readings](#) (byte loadcell_num, int n_readings)
Set the number of readings averaged for calibration of offset for a given [LoadCell](#).
- byte [get_tare_n_readings](#) (byte loadcell_num)
Get the number of readings averaged for calibration of offset for a given [LoadCell](#).
- void [set_scale_coeff_n_readings](#) (byte loadcell_num, int n_readings)
Set the number of readings averaged for calibration of scale coefficient for a given [LoadCell](#).
- byte [get_scale_coeff_n_readings](#) (byte loadcell_num)
Get the number of readings averaged for calibration of scale calibration for a given [LoadCell](#).
- void [set_weight_n_readings](#) (byte loadcell_num, int n_readings)
Set the number of readings averaged for weight measurements for a given [LoadCell](#).
- byte [get_weight_n_readings](#) (byte loadcell_num)
Get the number of readings averaged for measurements of a given [LoadCell](#)'s weight.
- void [set_all_loadcells_tare_n_readings](#) (int n_readings)
Set the number of readings averaged for calibration of offset for all [LoadCells](#).
- void [set_all_loadcells_scale_coeff_n_readings](#) (int n_readings)
Set the number of readings averaged for calibration of scale coefficient for all [LoadCells](#).
- void [set_all_loadcells_weight_n_readings](#) (int n_readings)
Set the number of readings averaged for weight measurements for all [LoadCells](#).
- void [set_mouse_weight](#) (float weight)
Set the value of member variable [mouse_weight](#).
- float [get_mouse_weight](#) ()
Get the value of member variable [mouse_weight](#).

- long [read_raw_average](#) (byte loadcell_num)
Read the output of the [LoadCell](#) and average `weight_n_readings` readings.
- long [read_tare_average](#) (byte loadcell_num)
Read the output of the [LoadCell](#) and average `tare_n_readings` readings.
- long [read_scale_coeff_average](#) (byte loadcell_num)
Read the output of the [LoadCell](#) and average `scale_coeff_n_readings` readings.
- double [get_raw_value](#) (byte loadcell_num)
Read the raw output of the [LoadCell](#) and subtracts the offset.
- float [get_weight](#) (byte loadcell_num)
Read the weight of the object currently on the [LoadCell](#).
- float [get_weight_with_auto_rec calibration](#) (byte loadcell_num, float threshold=0.33)
Read the weight of the object currently on the [LoadCell](#) and recalibrate the offset.
- float [mass_from_raw](#) (byte loadcell_num, long raw)
Calculate mass from a raw reading.
- void [tare](#) (byte loadcell_num)
Read the raw output of the [LoadCell](#) and set this value to the offset.
- void [begin](#) (byte loadcell_num, byte dout, byte pd_sck, byte gain=128)
Begin the communication with the HX711 for a given [LoadCell](#).
- bool [is_ready](#) (byte loadcell_num)
Verify if the HX711 is ready for a reading.
- void [wait_ready](#) (byte loadcell_num, unsigned long delay_ms=0)
Suspend the execution of the code until the HX711 becomes ready.
- bool [wait_ready_retry](#) (byte loadcell_num, int retries=3, unsigned long delay_ms=0)
Suspend the execution of the code until the HX711 becomes ready or for a given number of retries.
- bool [wait_ready_timeout](#) (byte loadcell_num, unsigned long timeout=1000, unsigned long delay_ms=0)
Suspend the execution of the code until the HX711 becomes ready or for a given timeout.
- void [power_up](#) (byte loadcell_num)
Wake up the chip after power down mode.
- void [power_down](#) (byte loadcell_num)
Wake up the chip after power down mode.
- byte [number_of_loadcells](#) ()
Get the number of [LoadCell](#) of the [LoadCellController](#).
- bool [is_loadcell_num_in_range](#) (byte loadcell_num)
Verify if a [LoadCell](#) number is in the range of the [LoadCellController](#).
- void [clear_eeprom](#) (int start=0, int end=1024)
Clear the EEPROM with 0 value.

Protected Member Functions

- void [easy_calibration_with_params](#) (byte loadcell_num, bool cal_tare, bool cal_scale)
Easy function managing the calibration of a [LoadCell](#).
- void [easy_save_to_persistent_memory_with_params](#) (byte loadcell_num, bool save_offset, bool save_scale)
Easy function managing the saving of calibration parameters to persistent memory.
- void [easy_read_from_persistent_memory_with_params](#) (byte loadcell_num, bool read_offset, bool read_↵ scale)
Easy function managing the reading of calibration from persistent memory.
- void [easy_start_with_questions](#) (byte loadcell_num, byte dout, byte pd_sck, byte gain=128)
Easy function for start with questions through the serial monitor.
- void [easy_handle_exceptions](#) (byte loadcell_num, bool calibrate_offset=false, bool calibrate_scale=false, bool read_offset_persistent_memory=false, bool read_scale_persistent_memory=false, bool save_offset_↵ _persistent_memory=false, bool save_scale_persistent_memory=false, float tare_offset=0, float scale_↵ coeff=0)
Easy function used to handle possible conflicts in [easy_start_with_params\(\)](#).

Protected Attributes

- `LoadCell * loadcells [10]`
Array containing the pointer of the [LoadCell](#) controlled by the [LoadCellController](#).
- `int n_loadcell = 0`
Member variable used to keep track of the number of LoadCells added.
- `float mouse_weight = 20`
Member variable used to store the weight in grams of the mouse in the cage controlled by The [LoadCellController](#).

9.2.1 Detailed Description

This library allows to control and manage many load cells with a [LoadCellController](#).

The [LoadCellController](#) library offers many features:

- Calibrate load cells, both parameters (offset and scale coefficient)
 - Member functions encapsulate the scripts needed for the calibration for both parameters. The interaction with the user is done inside the member functions.
- Save the calibration parameters to persistent memory

The library can automatically detect if an Arduino Uno (ATmega328p chip) or a FireBeetle (ESP8266 chip) is used. For Arduino Uno, EEPROM is used to store calibration parameters. For FireBeetle, SPIFFS is used.

The calibration parameters can be saved on the EEPROM after calibration for Atmega328p chip (Arduino Uno). These types of persistent memory are useful when it comes to saving because the saved values don't erase themselves when the power supply of the microcontroller is cut. The saved parameters can be reused at another moment. (tested on Arduino Uno and FireBeetle November 2023)

When it is wanted to reuse the saved parameters, it is important to add the load cell in the same order with the [add_loadcell\(\)](#) function. The calibration parameters are saved are accessible with the number of the [LoadCell](#).
- Read the calibration parameters from persistent memory (tested on Arduino Uno August 2023)
 - The calibration parameters can also be read from persistent memory after being saved.
- "Easy" start-up function that will handle everything related to the calibration and manage the calibration parameters.
 - The [LoadCellController](#) class offers ready-to-use "easy" functions that simplify the start-up of a load cell. Many scenarios are possible for the start-up of a load cell when it comes to calibration and the management of the calibration parameters.
 - A simple function [easy_start_with_params\(\)](#) takes in arguments many parameters to choose the appropriate option. Only a few parameters are needed.
 - This is useful for those who don't want to create their own sketch.
 - All the member functions that begin with "easy" are for easy start-up and don't implement any other functionalities than the other member functions. They only offer a "all-in-one" option for those who don't want to code.
- The [LoadCellController](#) class allows to access the load cells through the controller.

The load cells are added with the member function [add_loadcell\(\)](#). After that, all load cells are under the control of the [LoadCellController](#) and the member functions of the [LoadCell](#) class are accessed through the controller.

This access to the LoadCells through the [LoadCellController](#) requires to specify to [LoadCellController](#) a parameter corresponding to the number identifying the [LoadCell](#) concerned (1, 2, ...). Every member functions that have as input the number of a [LoadCell](#) (`n_loadcell`) will verify if the [LoadCell](#) number is in a coherent range according to the number of LoadCells added to the controller according to the member variable

add_loadcell() [2/2]

```
void LoadCellController::add_loadcell (
    LoadCell & loadcell,
    byte dout,
    byte sck,
    byte gain = 128)
```

Add a new [LoadCell](#) to the [LoadCellController](#) with pin specification.

This function adds a new [LoadCell](#) to the controller. It will add the pointer of the [LoadCell](#) object to the next index in the [loadcells](#) array and increase the value [n_loadcell](#) by 1 to keep track of the number of [LoadCell](#).

The first added [LoadCell](#) will be load cell 1, the second one will be load cell 2 and so on.

Parameters

<i>loadcell</i>	LoadCell object added to the LoadCellController .
<i>dout</i>	Digital pin (or analog) connected to the DOUT output pin of the HX711.
<i>sck</i>	Digital pin (or analog) connected to the SCK output of the HX711.
<i>gain</i>	Gain of the HX711. Default is 128.

calculate_scale_coeff()

```
float LoadCellController::calculate_scale_coeff (
    byte loadcell_num,
    float output,
    float mass)
```

calibrate_all_loadcells()

```
void LoadCellController::calibrate_all_loadcells ()
```

Calibrate the scale coefficient of every [LoadCells](#) added to the [LoadCellController](#).

This function interacts with the user to calibrate every [LoadCell](#) with known weights. The user is guided by the messages on the monitor to place the known weights on the load cell. The user is asked to send the weight value from the monitor. The scale coefficient are then saved to persistent memory.

calibrate_both_params()

```
void LoadCellController::calibrate_both_params (
    byte loadcell_num)
```

Proceed to the calibration of both parameters and set them for a [LoadCell](#).

This function begins the calibration of both offset and scale coefficient by calling the member functions [calibrate_tare_offset\(\)](#) and [calibrate_scale_coeff\(\)](#) that interact with the user to determine and set both calibration parameters.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

calibrate_scale_coeff()

```
void LoadCellController::calibrate_scale_coeff (
    byte loadcell_num)
```

Proceed to the calibration of the scale coefficient and set it for a [LoadCell](#).

This function use the member function [determine_scale_coeff\(\)](#) to get the scale coefficient from the calibration done by the user. This member function then sets this scale coefficient value to the member variable of the [LoadCell](#).

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

calibrate_tare_offset()

```
void LoadCellController::calibrate_tare_offset (
    byte loadcell_num)
```

Proceed to the calibration of the offset and set it for a [LoadCell](#).

This function use the member function [determine_offset\(\)](#) to get the offset from the calibration done by the user. This member function then sets this offset value to the member variable of the [LoadCell](#).

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

clear_eeprom()

```
void LoadCellController::clear_eeprom (
    int start = 0,
    int end = 1024)
```

Clear the EEPROM with 0 value.

This function writes a 0 at all index on the EEPROM for a given range.

Note

The `end` parameters has a default value of 1024 since it is the size of the Arduino Uno EEPROM. It might not be the case with other microcontroller.

Parameters

<i>start</i>	First index to clear. Default is 0
<i>end</i>	Last index to clear. Default is 1024

determine_offset()

```
float LoadCellController::determine_offset (
    byte loadcell_num)
```

Proceed to the calibration of the offset and return the offset value.

This function interacts with the user through the serial monitor to get an offset value for a given [LoadCell](#).

This member function asks to the user to remove any object from the load cell and send "t" with the serial monitor when it is done. After that, a output measurement is done with [read_tare_average\(\)](#) to get a raw output value. This value is returned.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

The offset value.

determine_scale_coeff()

```
float LoadCellController::determine_scale_coeff (
    byte loadcell_num)
```

Proceed to the calibration of the scale coefficient and return the scale coefficient value.

This function interacts with the user through the serial monitor to get a scale coefficient value for a given [LoadCell](#).

This member function asks the user to specify the number of reference weights that will be used to calibrate the load cell.

After that, the user is asked to put the first reference weight on the loadcell and send the weight value through the serial monitor. A weight measurement is done using the member function [read_scale_coeff_average\(\)](#). A couple for our calibration linear function -> $\text{output} = \text{scale_coeff} * \text{mass} + \text{offset}$ is obtained (known mass, measured output). The scale coefficient can now be calculated.

The process described above can be repeated for each reference weight to get a scale coefficient value.

The average of all scale coefficients is taken to get the finale value returned by the function.

Note

With this simple method, the scale coefficient depends of the offset. Finding both calibration parameters (offset and scale coefficient) at the same time with a curve fit algorithm could be more effective.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

The scale coefficient value.

easy_calibration_with_params()

```
void LoadCellController::easy_calibration_with_params (
    byte loadcell_num,
    bool cal_tare,
    bool cal_scale) [protected]
```

Easy function managing the calibration of a [LoadCell](#).

This function is managing the possibilities that come with the calibration of the tare offset and the scale coefficient. It is meant to be used inside the member function [easy_start_with_params\(\)](#).

The calibration process of both parameters (offset and scale coefficient) requires a computer to display information on the serial monitor and also to send value through it for the calibration.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>cal_tare</i>	Boolean telling if the offset should be calibrated.
<i>cal_scale</i>	Boolean telling if the scale coefficient should be calibrated.

easy_handle_exceptions()

```
void LoadCellController::easy_handle_exceptions (
    byte loadcell_num,
    bool calibrate_offset = false,
    bool calibrate_scale = false,
    bool read_offset_persistent_memory = false,
    bool read_scale_persistent_memory = false,
    bool save_offset_persistent_memory = false,
    bool save_scale_persistent_memory = false,
    float tare_offset = 0,
    float scale_coeff = 0) [protected]
```

Easy function used to handle possible conflicts in [easy_start_with_params\(\)](#).

This function manages the possible conflicts that can happen between the parameters passed to the [easy_start_with_params\(\)](#) member function. Since the user is specifying the starting parameters, it is probable that some are not possible simultaneously. This member function verifies all possible conflicts. If one is detected, the execution will be stopped with infinite while() loop, since it is not possible to do raise exceptions on Arduino (as I know). It is meant to be used inside the member function [easy_start_with_params\(\)](#).

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>calibrate_offset</i>	Boolean telling if the offset should be calibrated. Default is false.
<i>calibrate_scale</i>	Boolean telling if the scale coefficient should be calibrated. Default is false.
<i>read_offset_persistent_memory</i>	Boolean telling if the offset should be read from persistent_memory. Default is false.
<i>read_scale_persistent_memory</i>	Boolean telling if the scale coefficient should be read from persistent_↵ memory. Default is false.

<i>save_offset_persistent_memory</i>	Boolean telling if the offset should be saved to persistent_memory. Default is false.
<i>save_scale_persistent_memory</i>	Boolean telling if the scale coefficient should be saved to persistent_memory. Default is false.
<i>tare_offset</i>	Value of the tare offset. Default is 0.

The `tare_offset` default is 0 and is considered as the absence of a value. If no value wants to be specified, 0 should be given to the member function.

Parameters

<i>scale_coeff</i>	Value of the scale coefficient. Default is 0.
--------------------	---

The `scale_coeff` default is 0 and is considered as the absence of a value. If no value wants to be specified, 0 should be given to the member function.

easy_read_from_persistent_memory_with_params()

```
void LoadCellController::easy_read_from_persistent_memory_with_params (
    byte loadcell_num,
    bool read_offset,
    bool read_scale) [protected]
```

Easy function managing the reading of calibration from persistent memory.

This function is managing the possibilities that come with the reading from persistent memory of the tare offset and the scale coefficient of a [LoadCell](#). It is meant to be used inside the member function [easy_start_with_params\(\)](#).

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>read_offset</i>	Boolean telling if the offset should be read from persistent memory.
<i>read_scale</i>	Boolean telling if the scale coefficient should be read from persistent memory.

easy_save_to_persistent_memory_with_params()

```
void LoadCellController::easy_save_to_persistent_memory_with_params (
    byte loadcell_num,
    bool save_offset,
    bool save_scale) [protected]
```

Easy function managing the saving of calibration parameters to persistent memory.

This function is managing the possibilities that come with the saving of the tare offset and the scale coefficient of a [LoadCell](#). It is meant to be used inside the member function [easy_start_with_params\(\)](#).

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

<i>save_offset</i>	Boolean telling if the offset should be saved to persistent memory.
<i>save_scale</i>	Boolean telling if the scale coefficient should be saved to persistent memory.

easy_start_with_params()

```
void LoadCellController::easy_start_with_params (
    byte loadcell_num,
    byte dout,
    byte pd_sck,
    bool calibrate_offset = false,
    bool calibrate_scale = false,
    bool read_offset_persistent_memory = false,
    bool read_scale_persistent_memory = false,
    bool save_offset_persistent_memory = false,
    bool save_scale_persistent_memory = false,
    float tare_offset = 0,
    float scale_coeff = 0,
    byte gain = 128)
```

Easy function managing the start-up of a [LoadCell](#) with parameters as input.

This function, with only a few parameters as input, controls everything related to the start-up of a [LoadCell](#). The calibration of the offset and the scale coefficient and their saving/reading to/from the persistent memory. All possible conflicts between parameters are handled by the member function [easy_handle_exceptions\(\)](#).

This member function was created to simplify the start-up of a [LoadCell](#), especially for non-programmers. No additional functionalities that are not accessible from the other member functions are implemented (not labeled `easy_...`).

All the following "easy" member functions are used inside [easy_start_with_params\(\)](#) to dispatch to handling of all start-up scenarios:

- [easy_calibration_with_params\(\)](#)
- [easy_save_to_persistent_memory_with_params\(\)](#)
- [easy_read_from_persistent_memory_with_params\(\)](#)
- [easy_handle_exceptions\(\)](#)

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>dout</i>	Digital pin (or analog) connected to the DOUT output pin of the HX711.
<i>pd_sck</i>	Digital pin (or analog) connected to the SCK output of the HX711.
<i>calibrate_offset</i>	Boolean telling if the offset should be calibrated. Default is false.
<i>calibrate_scale</i>	Boolean telling if the scale coefficient should be calibrated. Default is false.
<i>read_offset_persistent_memory</i>	Boolean telling if the offset should be read from persistent memory. Default is false.
<i>read_scale_persistent_memory</i>	Boolean telling if the scale coefficient should be read from persistent memory. Default is false.

<i>save_offset_persistent_memory</i>	Boolean telling if the offset should be saved to persistent memory. Default is false.
<i>save_scale_persistent_memory</i>	Boolean telling if the scale coefficient should be saved to persistent memory. Default is false.
<i>tare_offset</i>	Value of the tare offset. Default is 0.
<i>scale_coeff</i>	Value of the scale coefficient. Default is 0.
<i>gain</i>	Gain of the HX711. Default is 128.

The `tare_offset` default is 0 and is considered as the absence of a value. If no value wants to be specified, 0 should be given to the member function.

The `scale_coeff` default is 0 and is considered as the absence of a value. If no value wants to be specified, 0 should be given to the member function.

easy_start_with_questions()

```
void LoadCellController::easy_start_with_questions (
    byte loadcell_num,
    byte dout,
    byte pd_sck,
    byte gain = 128) [protected]
```

Easy function for start with questions through the serial monitor.

This function is managing the possibilities that come with the start-up of a [LoadCell](#) with questions that are asked through the serial monitor of the Arduino.

Todo This could be done in the future to simplify the start-up of a [LoadCell](#) especially for non-programmers.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
<i>dout</i>	Pin connected to the DOUT output of the HX711.
<i>pd_sck</i>	Pin connected to the SCK output of the HX711.
<i>gain</i>	Gain of the HX711. Default is 128

The `gain` argument can take the following values: 128, 64 and 32. For a gain of 128 and 64, channel A of the HX711 should be used. For a gain of 32, channel B should be used.

get_offset_eeprom_adress()

```
int LoadCellController::get_offset_eeprom_adress (
    byte loadcell_num)
```

Get the EEPROM address for the offset of a [LoadCell](#).

This function returns the EEPROM address for the offset value for a given [LoadCell](#). The EEPROM addresses are based on the loadcell number.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

EEPROM address for the offset value.

Note

The EEPROM addresses are spaced along the EEPROM to prevent overlap between neighboring values.

get_offset_file_name()

```
String LoadCellController::get_offset_file_name (  
    byte loadcell_num)
```

Get the SPIFFS file name for the offset of a [LoadCell](#).

This function returns the offset file name for a given [LoadCell](#). The offset file name is based on the loadcell number.

Parameters

<code>loadcell_num</code>	Number of the LoadCell .
---------------------------	--

Returns

File name for the scale coefficient.

Note

Offset file name are formatted this way: "/offset{loadcell number}.txt". For example, the offset file name for [LoadCell](#) #1 is "/offset1.txt"

get_scale_coeff_eeprom_address()

```
int LoadCellController::get_scale_coeff_eeprom_address (  
    byte loadcell_num)
```

Get the EEPROM address for the scale coefficient of a [LoadCell](#).

This function returns the EEPROM address for the scale coefficient value for a given [LoadCell](#). The EEPROM addresses are based on the loadcell number.

Parameters

<code>loadcell_num</code>	Number of the LoadCell .
---------------------------	--

Returns

EEPROM address for the scale coefficient.

Note

The EEPROM addresses are spaced along the EEPROM to prevent overlap between neighboring values.

get_scale_coeff_file_name()

```
String LoadCellController::get_scale_coeff_file_name (
    byte loadcell_num)
```

Get the SPIFFS file name for the scale coefficient of a [LoadCell](#).

This function returns the scale coefficient file name for a given [LoadCell](#). The file name is based on the loadcell number.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

File name for the scale coefficient.

Note

Offset file name are formatted this way: "/offset{loadcell number}.txt". For example, the offset file name for [LoadCell](#) #1 is "/offset1.txt"

is_loadcell_num_in_range()

```
bool LoadCellController::is_loadcell_num_in_range (
    byte loadcell_num)
```

Verify if a [LoadCell](#) number is in the range of the [LoadCellController](#).

This function returns boolean telling whether a number of a [LoadCell](#) is in the range of the possible number of [LoadCell](#) for a [LoadCellController](#) according to his member variable [n_loadcell](#). This function is useful inside all member functions that loadcell_num as parameters to suspend the code execution if necessary.

Returns

Boolean telling if a [LoadCell](#) number is coherent for the [LoadCellController](#).

number_of_loadcells()

```
byte LoadCellController::number_of_loadcells ()
```

Get the number of [LoadCell](#) of the [LoadCellController](#).

This function returns the number of [LoadCell](#) that were added to the [LoadCellController](#) with the [add_loadcell\(\)](#) member function since the [n_loadcell](#) member variable is kept protected.

Returns

Number of [LoadCell](#).

read_all_scale_coeff_from_persistent_memory()

```
void LoadCellController::read_all_scale_coeff_from_persistent_memory ()
```

Read the scale coefficient of every LoadCells added to the [LoadCellController](#).

This function read the scale coefficient of every [LoadCell](#) from peristent memory. The scale coefficient is then set to the [LoadCell](#) class variable OFFSET through the controller with a setter. This function can be used in the start-up of LoadCells when the scale coefficients are saved to peristent memory.

read_offset_from_persistent_memory()

```
long LoadCellController::read_offset_from_persistent_memory (
    byte loadcell_num)
```

Read the offset of a given [LoadCell](#) from persistent memory.

This function reads the offset of a [LoadCell](#) from persistent memory, which was added to the controller with the member function [.For](#) a ATmega328p board, the EEPROM adress is obtained with the member function [get_offset_eeprom_adress\(\)](#). The process of assigning EEPROM addresses to [LoadCell](#) is completely hidden from the user to avoid problems. For a ESP32 board, the SPIFFS file name is obtained with the member function [get_offset_file_name\(\)](#). The same way, everything is hidden from the user.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

The offset of the given [LoadCell](#) obtained from persistent memory.

read_scale_coeff_from_persistent_memory()

```
float LoadCellController::read_scale_coeff_from_persistent_memory (
    byte loadcell_num)
```

Read the scale coefficient of a given [LoadCell](#) to persistent memory.

This function reads the offset of a [LoadCell](#) from persistent memory, which was added to the controller with the member function [add_loadcell\(\)](#). For a ATmega328p board, the EEPROM adress is obtained with the member function [get_offset_eeprom_adress\(\)](#). The process of assigning EEPROM addresses to [LoadCell](#) is completely hidden from the user to avoid problems. For a ESP32 board, the SPIFFS file name is obtained with the member function [get_offset_file_name\(\)](#). The same way, everything is hidden from the user.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

Returns

The scale coefficient of the given [LoadCell](#) obtained from persistent memory.

save_offset_to_persistent_memory()

```
void LoadCellController::save_offset_to_persistent_memory (
    byte loadcell_num)
```

Save the offset of a given [LoadCell](#) to persistent memory.

This function saves the offset member variable of a [LoadCell](#), which was added to the controller with the member function [add_loadcell\(\)](#). For a ATmega328p board, the EEPROM address is obtained with the member function [get_offset_eeprom_adress\(\)](#). The process of assigning EEPROM addresses to [LoadCell](#) is completely hidden from the user to avoid problems. For a ESP32 board, the SPIFFS file name is obtained with the member function [get_offset_file_name\(\)](#). The same way, everything is hidden from the user.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

save_scale_coeff_to_persistent_memory()

```
void LoadCellController::save_scale_coeff_to_persistent_memory (
    byte loadcell_num)
```

Save the scale coefficient of a given [LoadCell](#) to persistent memory.

This function saves the scale coefficient member variable of a [LoadCell](#), which was added to the controller with the member function [add_loadcell\(\)](#). For a ATmega328p board, the EEPROM address is obtained with the member function [get_offset_eeprom_adress\(\)](#). The process of assigning EEPROM addresses to [LoadCell](#) is completely hidden from the user to avoid problems. For a ESP32 board, the SPIFFS file name is obtained with the member function [get_offset_file_name\(\)](#). The same way, everything is hidden from the user.

Parameters

<i>loadcell_num</i>	Number of the LoadCell .
---------------------	--

tare_all_loadcells()

```
void LoadCellController::tare_all_loadcells (
    bool wait_for_user = true)
```

Tare all the LoadCells added to the [LoadCellController](#).

This function interacts with the user to tare every LoadCells added to the [LoadCellController](#) when he is ready. The user is asked to send 't' from the serial monitor when all load cells are empty. The taring of each will be done. A message will be printed on the monitor when done.

Parameters

<i>wait_for_user</i>	Boolean telling if the manual confirmation of the user is needed to tare the LoadCells
----------------------	--

9.2.4 Member Data Documentation

loadcells

```
LoadCell* LoadCellController::loadcells[10] [protected]
```

Array containing the pointer of the [LoadCell](#) controlled by the [LoadCellController](#).

The only moment this variable is modified is when a [LoadCell](#) is added with the [add_loadcell\(\)](#) member function. It goes up by one every time [add_loadcell\(\)](#) is called.

mouse_weight

```
float LoadCellController::mouse_weight = 20 [protected]
```

Member variable used to store the weight in grams of the mouse in the cage controlled by The [LoadCellController](#).

Default is 20 grams. This parameter can be used as a threshold for recalibration in [get_weight_with_auto_recibration](#)

n_loadcell

```
int LoadCellController::n_loadcell = 0 [protected]
```

Member variable used to keep track of the number of LoadCells added.

The only moment this variable is modified is when a [LoadCell](#) is added with the [add_loadcell\(\)](#) member function. It goes up by one every time [add_loadcell\(\)](#) is called.